

Data Mining Practice10

Team 2

양승렬 2200409

최서헌 22100133

문벼리 22000245

Task 1

Task 2

Task 3

Task4

01 | 데이터 전처리

02 | 모델링

Task5

Task 1

- Explain how to calculate the decision boundary of the original scale on page 37 and draw a graph similar to the one on page 38.

[row data]

이름	타입	설명
exam1	double	첫번째 시험 점수
exam2	double	두번째 시험 점수
admitted	integer	합격 여부

Base code

- 데이터 전처리 및 스케일링
- 비용 함수와 가설 함수 정의
- 그래디언트 디센트로 파라미터 업데이트
- 학습 곡선 및 결정 경계 시각화

```

#데이터 로드
admission <- read.csv('ex2data1.txt', header = F,
  col.names = c('exam1','exam2','admitted'))

#초기 theta값
theta_vector <- c(0,0,0)

feature.df <- admission %>%
  mutate(bias = 1) %>%
  select(bias, exam1, exam2)
labelVector <- admission$admitted

#sigmoid함수
sigmoid <- function(x){
  1 / (1 + exp(-x))
}

#가설함수
h <- function(x, theta_vector){
  sigmoid(theta_vector %*% x)
}

#cost함수
costFunction <- function(theta_vector){
  hx <- apply(feature.df, 1, function(x){h(x, theta_vector)})
  costV <- labelVector * log(hx) + (1-labelVector) * log(1-hx)
  -mean(costV)
}
costFunction(theta_vector)

feature.df[,2:3] <- sapply(feature.df[,2:3],
  function(x){(x-mean(x))/sd(x)})
summary(feature.df)

#gradient descent
theta_vector <- c(0, 0, 0)
costDF <- data.frame(iter = 0,

```

```

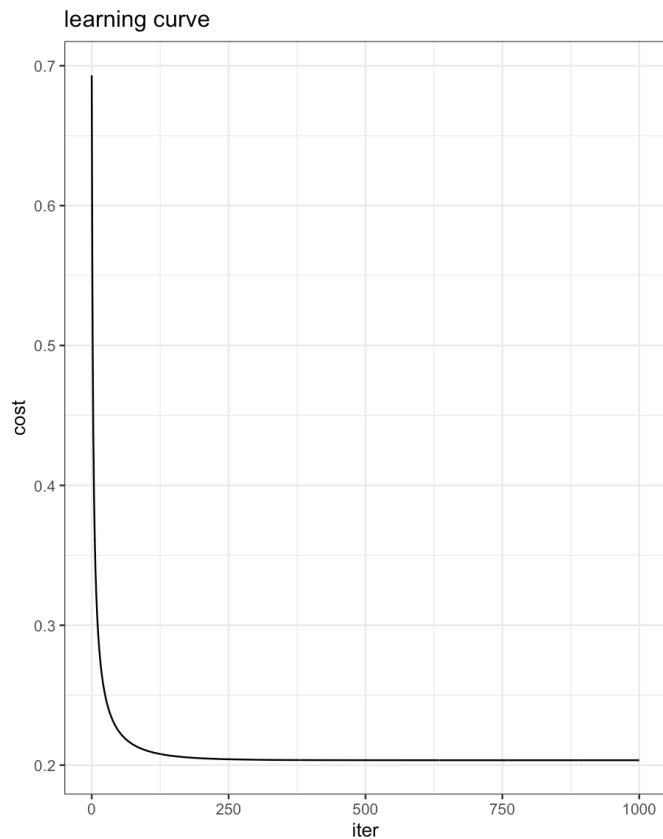
        t0 = theta_vector[1],
        t1 = theta_vector[2],
        t2 = theta_vector[3],
        cost = costFunction(theta_vector))

## find optimal theta
num_iter <- 1000
m <- nrow(feature.df)
alpha = 1
for(i in 1:num_iter){
  theta_update <-
    (t(as.matrix(feature.df)) %*% (apply(feature.df, 1,
      function(x) { h(x, theta_vector) }) -
      labelVector)) / m * alpha
  theta_vector <- theta_vector - theta_update[,1]
  costDF <- rbind(costDF, c(i, theta_vector, costFunction(theta_vector)))
}

#learning Curve
costDF %>% ggplot(aes(x = iter, y= cost)) +
  geom_line() +
  ggtitle('learning curve') + theme_bw()

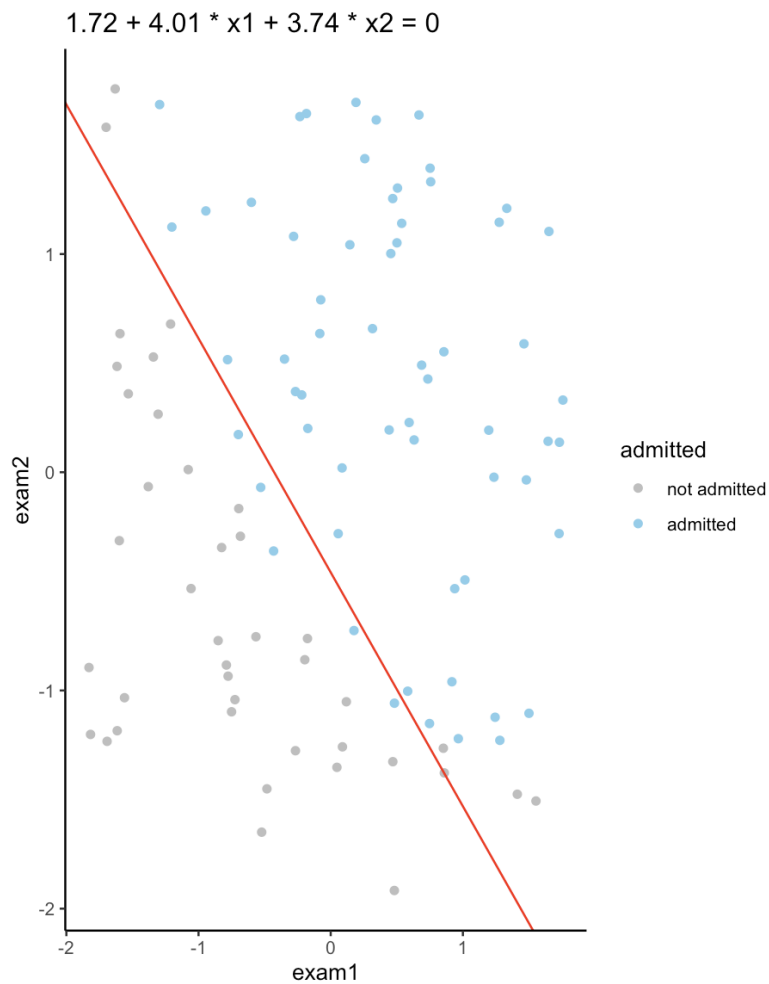
```

Learning Curve



- 250회 반복 → 수렴

```
slope <- - theta_vector[2] / theta_vector[3]
bias <- - theta_vector[1] / theta_vector[3]
ggplot(feature.df, aes(x=exam1, y=exam2)) +
  geom_point(aes( col = as.factor(labelVector))) +
  geom_abline(slope = slope, intercept = bias, col = 'red')+
  scale_color_manual('admitted', values = c('grey','skyblue'))+
  labels = c('not admitted','admitted'))+
  ggtitle(sprintf("%.2f + %.2f * x1 + %.2f * x2 = 0",
    theta_vector[1],theta_vector[2],theta_vector[3]))+
  theme_classic()
costFunction(theta_vector)
```



- Problem Solving

```
theta_origin <- (theta_vector[1] - mu1 / sigma1, theta_vector[2] - mu2 / sigma2)

slope <- theta_origin[1] / theta_origin[2]
bias <- theta_origin[2] / theta_origin[1]
```

표준화된 exam1, exam2

$$x'_1 = \frac{x_1 - \mu_1}{\sigma_1}$$

$$x'_2 = \frac{x_2 - \mu_2}{\sigma_2}$$

로지스틱 회귀(Logistic Regression)의 결정 경계 방정식

$$\theta_0 + \theta_1 \cdot x'_1 + \theta_2 \cdot x'_2 = 0$$

로지스틱 회귀의 결정 경계 방정식에 대입

$$\theta_0 + \theta_1 \cdot \frac{x_1 - \mu_1}{\sigma_1} + \theta_2 \cdot \frac{x_2 - \mu_2}{\sigma_2} = 0$$

식 정리

$$\left(\theta_0 - \frac{\theta_1 \cdot \mu_1}{\sigma_1} - \frac{\theta_2 \cdot \mu_2}{\sigma_2}\right) + \frac{\theta_1}{\sigma_1} \cdot x_1 + \frac{\theta_2}{\sigma_2} \cdot x_2 = 0$$

original bias

$$\theta'_0 = \theta_0 - \frac{\theta_1 \cdot \mu_1}{\sigma_1} - \frac{\theta_2 \cdot \mu_2}{\sigma_2}$$

original theta

$$\theta'_1 = \frac{\theta_1}{\sigma_1}, \quad \theta'_2 = \frac{\theta_2}{\sigma_2}$$

- 정규화된 파라미터를 원래 스케일로 변환 후 결정 경계 시각화

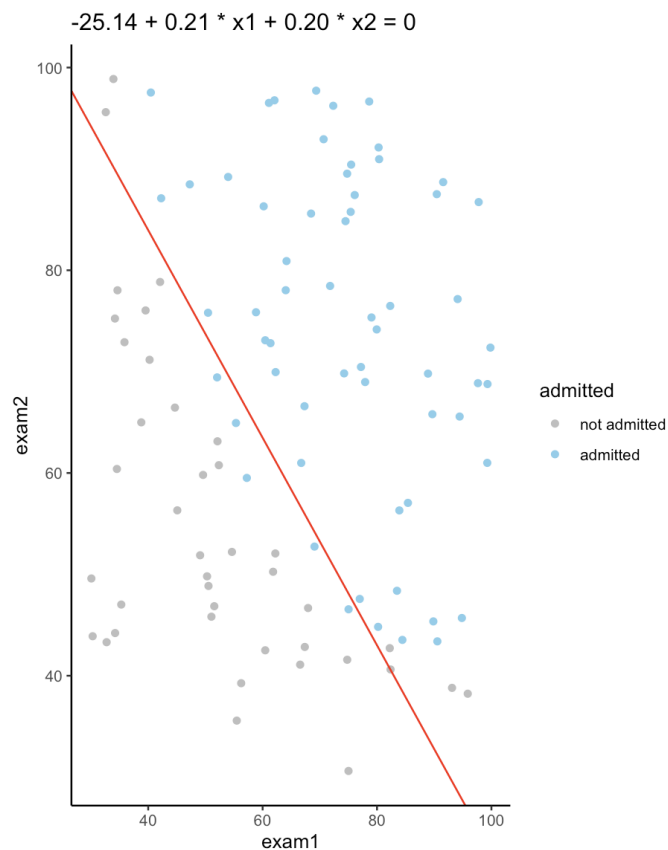
```
#평균 및 표준편차 할당
mu1 <- mean(admission$exam1)
mu2 <- mean(admission$exam2)
sig1 <- sd(admission$exam1)
sig2 <- sd(admission$exam2)

# theta_vector를 원래 스케일로 변환
theta_origin <- c(
  theta_vector[1] - (mu1 / sig1) * theta_vector[2] -
  (mu2 / sig2) * theta_vector[3],
  theta_vector[2] / sig1,
  theta_vector[3] / sig2
)

# 경계선의 기울기와 절편 계산
slope <- -theta_origin[2] / theta_origin[3]
bias <- -theta_origin[1] / theta_origin[3]
```

```
ggplot(admission, aes(x=exam1, y=exam2)) +
  geom_point(aes( col = as.factor(labelVector))) +
  geom_abline(slope = slope, intercept = bias, col = 'red')+
  scale_color_manual('admitted', values = c('grey','skyblue'))
  labels = c('not admitted','admitted'))+
  ggtitle(sprintf("%.2f + %.2f * x1 + %.2f * x2 = 0",
    theta_origin[1],theta_origin[2],theta_origin[3]))+
  theme_classic()
```

• Result



Task 2

```
admission <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학
기/데이터 마이닝 실습/Data-Mining-Practicum/myR/ex2data1.txt",
```

```
header = F,
col.names = c('exam1', 'exam2', 'admitted'))
```

다시, 데이터를 가져온다.

데이터는 exam1 과 exam2, 그리고 admitted(입학 여부)를 가진 데이터프레임이며, 우리는 이 exam1과 exam2변수를 통해 admitted를 예측하고자 한다

```
admission <- admission %>%
  mutate(exam1_sq = exam1^2,
         exam2_sq = exam2^2,
         exam1_exam2 = exam1 * exam2)
```

다항 특징을 생성한다, mutate로 새로운 변수들을 만들며, 기존 변수인 exam1과 exam2를 제공하거나, 두 변수를 곱한 값을 새로운 변수로 만든다. 이 feature은 기존 특징 외 새로운 추가 특징으로 사용된다, 로지스틱 회귀는 선형적인 분류 경계를 만드는데, 다항 특징(비선형 특징)을 추가하면 모델이 더 복잡한 결정 경계를 학습할 수 있게 되어 데이터가 더 잘 분류될 가능성이 있다.

예를 들어,

$exam1^2$ 이나 $exam2^2$ 는 비선형성을 추가해 두 변수의 상호작용과 비선형 패턴을 학습하도록 돕는다.

```
# 추가 변수를 포함한 특징 데이터프레임 생성 및 정규화
feature.df <- admission %>%
  mutate(bias = 1) %>%
  select(bias, exam1, exam2, exam1_sq, exam2_sq, exam1_exam2)
```

이제 추가한 변수들을 사용하여 feature df를 만든다.

이 feature df에 bias라는 상수 열을 추가하여 절편을 고려할 수 있도록 한다. 이후 select로 모델에 사용할 특징을 선택한다.

로지스틱 회귀 모델에서는 절편을 학습할 수 있어야 하므로 bias항을 추가한다.

```
feature.df[, 2:6] <- sapply(feature.df[, 2:6], function(x)
(x - mean(x)) / sd(x))
```

각 feature들을 정규화한다. 평균을 0, 분산을 1로 바꾸어 경사하강법이 더욱 효율적으로 작동되도록 한다. (이전 선형회귀에서와 같은 이유)

```
labelVector <- admission$admitted
```

종속 변수(레이블)을 저장한다. 모델의 예측 값과 실제 값을 비교하기 위해 이후 예측값과 labelvector과 비교하여 성능을 확인하게 된다

```
sigmoid <- function(x) {
  1 / (1 + exp(-x))
}
```

다음 시그모이드 함수를 정의한다. 로지스틱 회귀는 이진 분류 모델로, 예측값이 확률로 표현되어야 한다. 시그모이드 함수는 입력값을 0과 1 사이의 값으로 변환하여 특정 사건이 일어날 확률을 나타낸다. 이 확률을 기준으로 모델은 클래스(입학 여부)를 예측하게 된다.

(시그모이드 함수에 대한 추가적인 내용 첨부 혹은 Task1에서 명시)

```
h <- function(x, theta_vector) {
  sigmoid(theta_vector %*% x)
}
```

가설 함수를 정의한다. 위에서 정한 sigmoid함수를 이용해서 주어진 `theta_vector` 와 `x` 에 대해 시그모이드 함수를 적용하여 예측 확률을 계산한다. `theta_vector %*% x` 는 특징의 가중합을 의미하며, 이 값을 시그모이드 함수에 넣어 확률을 계산한다. 이를 통해 각 데이터 포인트가 입학할 확률을 구할 수 있다.

`theta_vector`는 각 특징에 대해 대한 가중치를 의미하며, 현재 사용된 bias, exam1, exam2, exam1_sq, exam2_sq, exam1_exam2에 대해 $\text{vector} = \{\theta_0, \theta_1, \theta_2, \theta_3, \theta_4, \theta_5\}$ 의 형태로 구성됨

이때 θ_0 은 bias로 절편에 대한 값이 들어가며, 그 외 θ 들은 각 feature의 가중치이다.

이러한 벡터값들은 모델이 학습하는 과정에서 최적의 값을 찾아가며 각 feature의 중요도를 의미한다.

이 theta_vector와 내적하는 x는 특정 데이터포인트의 실제 값으로서 로지스틱 회귀에서 계산하게 되는 가설함수인 $\theta_0 + \theta_1 * x_1 \dots$ 의 계산식을 따르게 된다.

```
costFunction <- function(theta_vector) {  
  hx <- apply(feature.df, 1, function(x) { h(x, theta_vector) })  
  costV <- labelVector * log(hx) + (1 - labelVector) * log(1 - hx)  
  -mean(costV)  
}
```

다음으로 예측 결과와 이전 labelVector에 저장한 실제 값의 차이를 측정하는 cost함수를 만든다

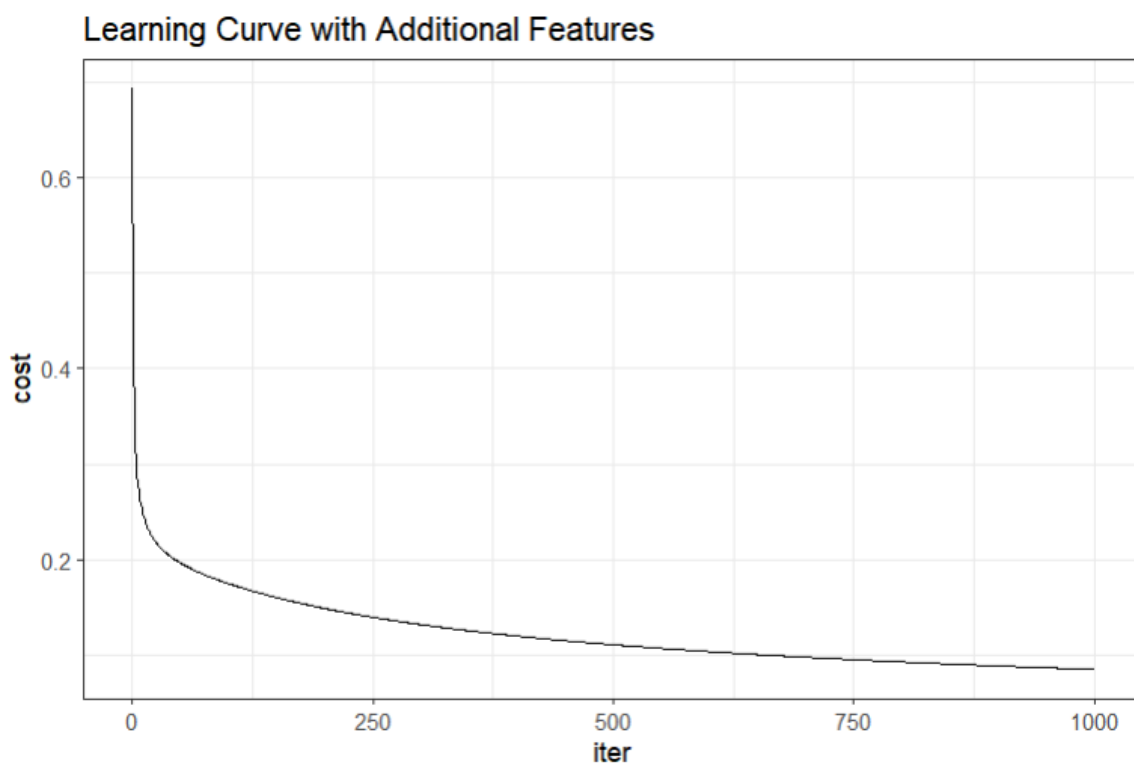
```
num_iter <- 1000  
m <- nrow(feature.df)  
alpha <- 1  
costDF <- data.frame(iter = 0,  
                      t0 = theta_vector[1],  
                      t1 = theta_vector[2],  
                      t2 = theta_vector[3],  
                      t3 = theta_vector[4],  
                      t4 = theta_vector[5],  
                      t5 = theta_vector[6],  
                      cost = costFunction(theta_vector))  
  
for (i in 1:num_iter) {  
  theta_update <- (t(as.matrix(feature.df)) %*% (apply(feature.df, 1, function(x) { h(x, theta_vector) }) - labelVector)) / m * alpha  
  theta_vector <- theta_vector - theta_update[,1]  
  costDF <- rbind(costDF, c(i, theta_vector, costFunction(theta_vector)))  
}
```

```
heta_vector)))
}
```

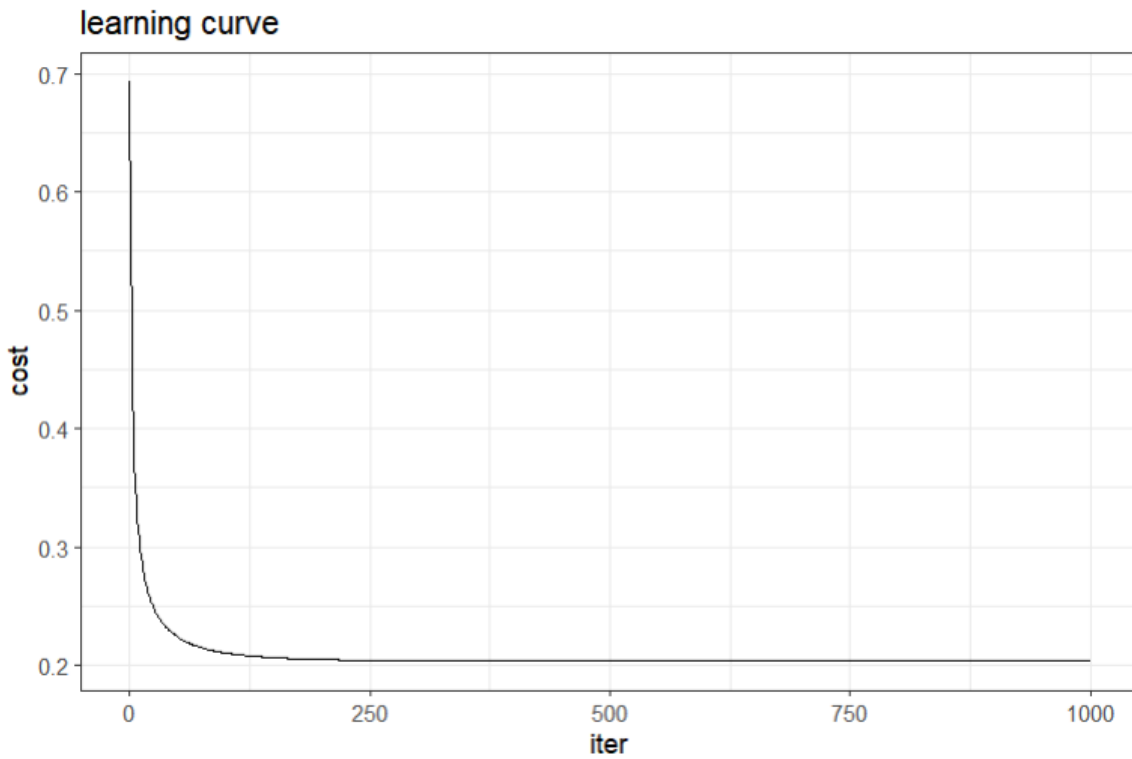
다음으로 경사하강법을 통해 cost를 최소화하는 theta_vector을 찾는다. 이 과정을 통해 최적의 theta_vector을 찾아 구하고 데이터에 적합한 모델이 된다.

학습 곡선 시각화

```
ggplot(costDF, aes(x = iter, y = cost)) +
  geom_line() +
  ggtitle('Learning Curve with Additional Features') +
  theme_bw()
```



학습 시행 횟수에 따른 cost값의 변화를 다음과 같이 표현하였다. 시행횟수가 250이 되기 전에 급격히 오차가 줄어들고 점점 수렴하다 시행이 끝나는 1000회에서 가장 낮다. 하지만 수렴하지는 않는 것 같다.

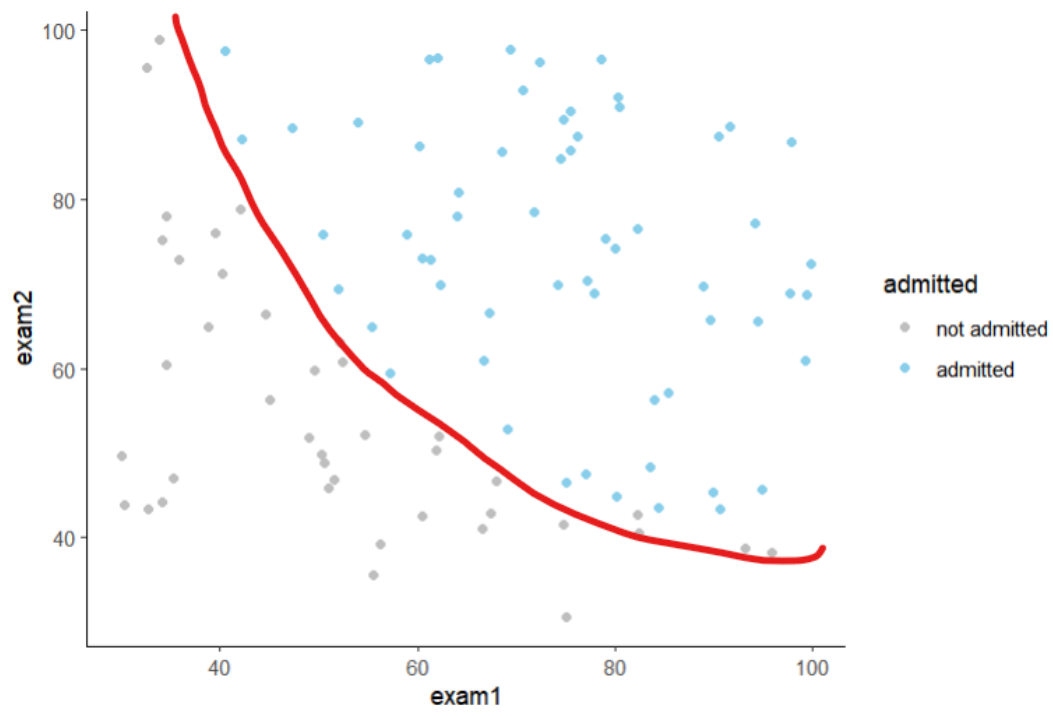


입력변수를 추가하기 전과 비교하면 오차가 더욱 줄어들었고 그 대신 수렴하지는 않았다.

Task 3

- 문제2에서 학습한 모델의 decision boundary의 식을 표현하고 38페이지와 같은 형식의 그래프로 그려보아라.

decision boundary 결정경계는 분류 모델에서 데이터를 분류하기 위한 경계선을 말한다.



exam1 과 exam2로 합격과 불합격을 나눈 그래프에서는 다음과 같은 붉은 선이 데이터를 분류하는 경계선이 될 수 있는데, 이를 구하기 위한 식과 실제 그래프상에서 표현이 가능하다.

로지스틱 회귀에서 결정 경계는 $\theta_0 + \theta_1 \times x_1 + \theta_2 \times x_2 = 0$

이 식으로 계산이 가능하다. 이 식에 우리가 구한 theta_vector을 대입하면

```
printf("%.2f + %.2f * x1 + %.2f * x2 + %.2f * x1^2 + %.2f * x2^2 + %.2f * x1*x2 = 0",
        theta_vector[1], theta_vector[2], theta_vector[3],
        theta_vector[4], theta_vector[5], theta_vector[6])
```

```
[1] "4.68 + 3.09 * x1 + 2.61 * x2 + -2.73 * x1^2 + -2.00 * x2^2 + 9.49 * x1*x2 = 0"
```

이러한 결정경계 식이 완성된다.

이제 이를 계산하고 시각화를 하면 다음과 같다

```
x1_vals <- seq(min(admission$exam1), max(admission$exam1),
length.out = 100)
x2_vals <- seq(min(admission$exam2), max(admission$exam2),
length.out = 100)
grid <- expand.grid(exam1 = x1_vals, exam2 = x2_vals)
```

우선 우리가 사용한 입력값이 다양하기 때문에, 실제 exam1과 exam2의 범위 안에서 100개의 값을 생성하고, 각각의 최소, 최대값을 가져와 $x_1 \times x_2$ 에 저장한다. 이후 x_1 과 x_2 의 모든 조합을 가지고 grid라는 데이터 프레임을 만든다

	exam1	exam2
1	30.05882	30.60326
2	30.76356	30.60326
3	31.46830	30.60326
4	32.17304	30.60326
5	32.87777	30.60326
6	33.58251	30.60326

다음과 같은 총 10,000개의 값이 포함되었음 이 데이터프레임은

exam1과 exam2가 가장 적을 때 부터 많을 때 까지의 결정 경계를 확인하기 위한 값들이 포함되어 있다.

```
grid <- grid %>%
  mutate(exam1_sq = exam1^2,
         exam2_sq = exam2^2,
         exam1_exam2 = exam1 * exam2)
```

이제 grid에서 입력값인 x_1^2, x_2^2, x_1x_2 에 대해 새로운 변수를 만든다

```
grid_norm <- as.data.frame(sapply(grid, function(x) (x - mean(x)) / sd(x))))
grid_norm <- cbind(bias = 1, grid_norm) # 편향 추가
```

feature df와 동일하게, 정규화를 거친 후

```
prob_vals <- apply(grid_norm, 1, function(x) h(x, theta_vector))
```

각 grid 포인트에 대해 예측을 수행한다. 이 예측값은 앞서 만든 theta_vector과 동일하다 (최적화된 값)

```
grid$prob <- prob_vals
```

이제 예측 결과를 grid에 prob열에 저장한다.

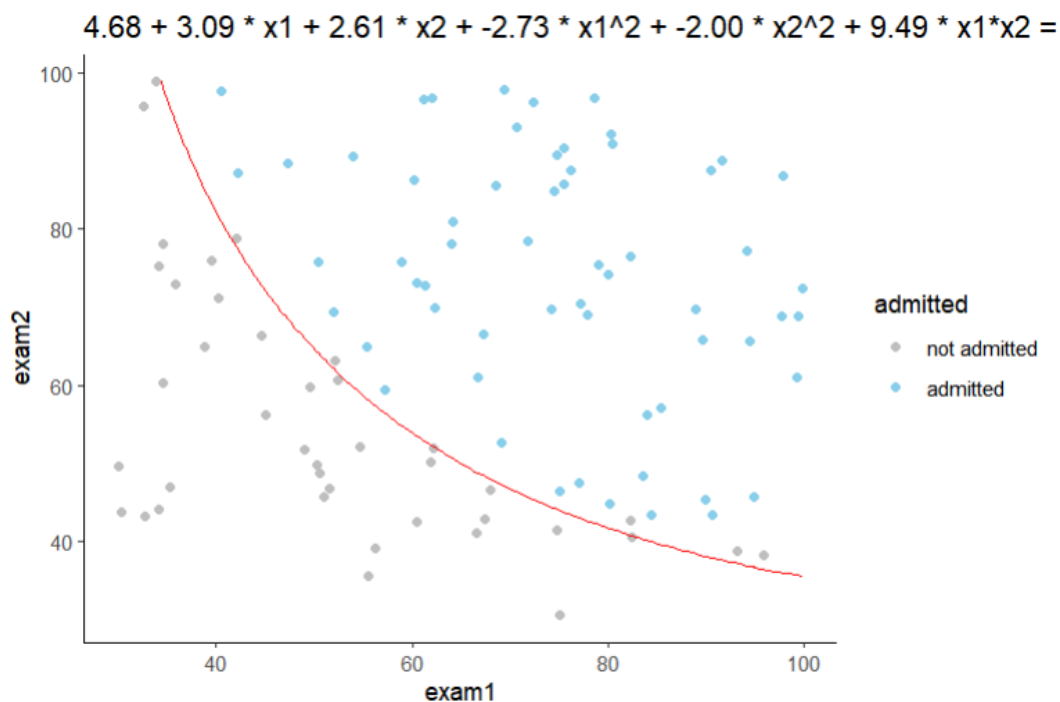
	exam1	exam2	exam1_sq	exam2_sq	exam1_exam2	prob
1	30.05882	30.60326	903.5328	936.5597	919.8981	3.150922e-07
2	30.76356	30.60326	946.3966	936.5597	941.4653	3.740741e-07
3	31.46830	30.60326	990.2538	936.5597	963.0326	4.436464e-07
4	32.17304	30.60326	1035.1042	936.5597	984.5999	5.256243e-07
5	32.87777	30.60326	1080.9480	936.5597	1006.1672	6.221186e-07
6	33.58251	30.60326	1127.7851	936.5597	1027.7344	7.355805e-07
7	34.28725	30.60326	1175.6154	936.5597	1049.3017	8.688533e-07
8	34.99199	30.60326	1224.4391	936.5597	1070.8690	1.025232e-06
9	35.69672	30.60326	1274.2561	936.5597	1092.4363	1.208528e-06
10	36.40146	30.60326	1325.0664	936.5597	1114.0035	1.423150e-06
11	37.10620	30.60326	1376.8701	936.5597	1135.5708	1.674187e-06
12	37.81094	30.60326	1429.6670	936.5597	1157.1381	1.967507e-06
13	38.51568	30.60326	1483.4572	936.5597	1178.7053	2.309872e-06
14	39.22041	30.60326	1538.2408	936.5597	1200.2726	2.709061e-06
15	39.92515	30.60326	1594.0177	936.5597	1221.8399	3.174014e-06
16	40.62989	30.60326	1650.7878	936.5597	1243.4072	3.714995e-06

이제, 각 feature별 값과, 예측값인 prob값이 저장되어 있다.

```
# 시각화 - 결정 경계를 등고선으로 그리기
ggplot(admission, aes(x = exam1, y = exam2, color = as.factor(admitted))) +
  geom_point() +
  geom_contour(data = grid, aes(x = exam1, y = exam2, z = prob),
    breaks = 0.5, color = 'red') +
  scale_color_manual('admitted', values = c('grey', 'skyblue'))
```

```
e'), labels = c('not admitted', 'admitted')) +
  ggtitle(sprintf("%.2f + %.2f * x1 + %.2f * x2 + %.2f * x1
^2 + %.2f * x2^2 + %.2f * x1*x2 = 0",
                  theta_vector[1], theta_vector[2], theta_v
ector[3], theta_vector[4], theta_vector[5], theta_vector
[6])) +
  theme_classic()
```

이제 exam1과 exam2에 대한 결정경계식을 적용한 시각화가 가능하다



Task4

- Kaggle Titanic Dataset을 학습하여 생존 여부를 판단하는 logistic model을 만들라. (train.csv 파일 사용) 강의 및 실습에서 연습한 것처럼 logistic regression은 패키지를 사용하지 않고 직접 구현하고 학습하는 알고리즘을 수행해야한다.

01 | 데이터 전처리

1. 데이터 설명

Name	Type	Description
PassengerId	integer	각 승객을 고유하게 식별하는 ID

Survived	integer	생존 여부
Pclass	integer	승객이 탑승한 클래스 등급 (1 = 1등석, 2 = 2등석, 3 = 3등석)
Name	character	승객의 이름
Sex	character	승객의 성별
Age	double	승객의 나이
SibSp	integer	함께 탑승한 형제자매 또는 배우자의 수
Parch	integer	함께 탑승한 부모 또는 자녀의 수
Ticket	character	승객의 티켓 번호
Fare	double	승객이 지불한 운임 요금
Cabin	character	승객의 선실 번호
Embarked	character	승객이 탑승한 항구 (C = Cherbourg, Q = Queenstown, S = Southampton)

2. 필요한 열 선택

```
train <- train %>%
  select(Survived, Pclass, Sex, SibSp, Parch, Fare, Embarked)
```

Name	전처리
Survived	목적 변수
Pclass	범주화 필요
Sex	바로 사용 가능 (0,1 이진 변수이기 때문에)
SibSp	바로 사용 가능
Parch	바로 사용 가능
Fare	바로 사용 가능
Embarked	범주화 필요

▼ 삭제 이유

1. **PassengerId** : 고유 식별자이므로 의미 없음
2. **Name** : 고유성이 높아 의미 있는 정보 도출이 어려움
3. **Age** : 결측치가 많음 (train-177, test-86)
4. **Ticket** : 고유 개수가 많고 일관성이 없어 의미 없음

5. **Cabin**: 고유 개수가 많고 불규칙적이라 의미 없음

3. 데이터 전처리

[one-hot encoding]

```
library(caret)

## train
# Sex 변수 처리
train$Sex <- ifelse(train$Sex == "male", 1, 0)

# Pclass를 범주형으로 변환
train$Pclass <- as.factor(train$Pclass)

# dummyVars 생성
dummy <- dummyVars(~ Pclass + Embarked, data = train)

# one-hot encoding 적용
encoded_df <- predict(dummy, newdata = train)

# 원본 데이터와 결합
train <- cbind(train, encoded_df)

# 원래의 범주형 변수 삭제
train <- train %>% select(-Pclass, -Embarked)
```

- label encoding을 사용하면 클래스 간의 순서나 크기를 학습 할 수 있기에 one-hot encoding 사용

Pclass	Embarked
3	S
1	C
3	S
1	S
3	S

Pclass.1	Pclass.2	Pclass.3	EmbarkedC	EmbarkedQ	EmbarkedS
0	0	1	0	0	1
1	0	0	1	0	0
0	0	1	0	0	1
1	0	0	0	0	1
0	0	1	0	0	1

[표준화]

```
# 표준화를 위해 각각의 평균, 표준편차 저장
SibSp_mean <- mean(train$SibSp, na.rm = T)
SibSp_sd <- sd(train$SibSp, na.rm = T)

Parch_mean <- mean(train$Parch, na.rm = T)
Parch_sd <- sd(train$Parch, na.rm = T)

Fare_mean <- mean(train$Fare, na.rm = T)
Fare_sd <- sd(train$Fare, na.rm = T)

# train 표준화
train$SibSp_norm <- (train$SibSp - SibSp_mean) / SibSp_sd
train$Parch_norm <- (train$Parch - Parch_mean) / Parch_sd
train$Fare_norm <- (train$Fare - Fare_mean) / Fare_sd

# 표준화 이전 열 제거
train <- train[, -c(3:5)]
```

- SibSp, Parch, Fare에 대해 표준화 진행 (다른 변수는 이미 0,1)

SibSp_norm	Parch_norm	Fare_norm
Min. :-0.4743	Min. :-0.4734	Min. :-0.64806
1st Qu.:-0.4743	1st Qu.:-0.4734	1st Qu.:-0.48887
Median :-0.4743	Median :-0.4734	Median :-0.35719
Mean : 0.0000	Mean : 0.0000	Mean : 0.00000
3rd Qu.: 0.4325	3rd Qu.:-0.4734	3rd Qu.:-0.02423
Max. : 6.7804	Max. : 6.9702	Max. : 9.66174

02 | 모델링

1. theta vector 초기화

```
theta_vector <- rep(0, 11)
```

```
theta_vector <- matrix(theta_vector, ncol = 1)
```

```
[11, 1]
```

2. feature matrix 생성

```
## train
train_feature.df <- train %>%
  mutate(bias = 1) %>%
  select(bias, colnames(train)[2:11])

train_feature_matrix <- as.matrix(train_feature.df)
```

bias	Sex	Pclass.1	Pclass.2	Pclass.3	EmbarkedC	EmbarkedQ	EmbarkedS	SibSp_norm	Parch_norm	Fare_norm
1	1	0	0	1	0	0	1	0.4325504	-0.4734077	-0.50216314
1	0	1	0	0	1	0	0	0.4325504	-0.4734077	0.78640362
1	0	0	0	1	0	0	1	-0.4742788	-0.4734077	-0.48857985
1	0	1	0	0	0	0	1	0.4325504	-0.4734077	0.42049407
1	1	0	0	1	0	0	1	-0.4742788	-0.4734077	-0.48606443
1	1	0	0	1	0	1	0	-0.4742788	-0.4734077	-0.47784805
1	1	1	0	0	0	0	1	-0.4742788	-0.4734077	0.39559138
1	1	0	0	1	0	0	1	2.2462089	0.7671990	-0.22395734
1	0	0	0	1	0	0	1	-0.4742788	2.0078057	-0.42401800
1	0	0	1	0	1	0	0	0.4325504	-0.4734077	-0.04293139
1	0	0	0	1	0	0	1	0.4325504	0.7671990	-0.31199715
1	0	1	0	0	0	0	1	-0.4742788	-0.4734077	-0.11378180
1	1	0	0	1	0	0	1	-0.4742788	-0.4734077	-0.48606443
1	1	0	0	1	0	0	1	0.4325504	5.7296258	-0.01869881
1	0	0	0	1	0	0	1	-0.4742788	-0.4734077	-0.49000459
1	0	0	1	0	0	0	1	-0.4742788	-0.4734077	-0.32608352
1	1	0	0	1	0	1	0	3.1530382	0.7671990	-0.06196409
1	1	0	1	0	0	0	1	-0.4742788	-0.4734077	-0.38645367
1	0	0	0	1	0	0	1	0.4325504	-0.4734077	-0.28583675

```
[891, 11]
```

3. target feature 설정

```
train_labelVector <- train$Survived
```

4. 시그모이드 함수 정의

$$g(z) = \frac{1}{1 + e^{-z}}$$

```
sigmoid <- function(x) {
  1 / (1 + exp(-x))
}
```

5. 가설 함수 정의

$$h_{\theta}(x) = g(\theta^T x)$$

```
h <- function(x, theta_vector){
  sigmoid(x %*% theta_vector)
}
```

6. 비용 함수 정의

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

```
costFunction <- function(theta_vector) {
  hx <- h(train_feature_matrix, theta_vector)
  costV <- train_labelVector * log(hx) +
    (1-train_labelVector) * log(1-hx) -mean(costV)
}
```

7. CostDF 생성

```
costDF <- data.frame(
  iter = 0,
  t0 = theta_vector[1],
  t1 = theta_vector[2],
```

```

t2 = theta_vector[3],
t3 = theta_vector[4],
t4 = theta_vector[5],
t5 = theta_vector[6],
t6 = theta_vector[7],
t7 = theta_vector[8],
t8 = theta_vector[9],
t9 = theta_vector[10],
t10 = theta_vector[11],
cost = costFunction(theta_vector)
)

```

8. 하이퍼 파라미터 설정

```

num_iter <- 1000
m <- nrow(train_feature.df)
alpha = 1

```

9. 학습

$$\theta_n := \theta_n - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

```

for(i in 1:num_iter) {
  # theta 업데이트 계산
  theta_update <- (t(as.matrix(train_feature_matrix)) %*%
    (apply(train_feature_matrix, 1,
    function(x) { h(x, theta_vector) }) -
    train_labelVector)) / m * alpha

  # theta_vector 업데이트
  theta_vector <- theta_vector - theta_update[,1]

  # costDF에 현재 반복의 값 추가
  costDF <- rbind(costDF, c(i, theta_vector,

```

```

    costFunction(theta_vector)))
}

theta_vector

```

결과 해석을 위한 역변환

정규화항 조정

```

theta_origin <- c(
  theta_vector[1] - (theta_vector[9] * SibSp_mean / SibSp_
    (theta_vector[10] * Parch_mean / Parch_sd) -
    (theta_vector[11] * Fare_mean / Fare_sd),
  theta_vector[9] / SibSp_sd,
  theta_vector[10] / Parch_sd,
  theta_vector[11] / Fare_sd
)

```

```

theta_vector[1,1] <- theta_origin[1]
theta_vector[9,1] <- theta_origin[2]
theta_vector[10,1] <- theta_origin[3]
theta_vector[11,1] <- theta_origin[4]

```

오즈 비율 조정

```

odds_ratios <- exp(theta_vector)

```

feature	theta	정규화 조정	오즈 조정
bias	0.98	1.03	2.81
Sex	-2.72	-2.72	0.07
Pclass.1	1.04	1.04	2.84
Pclass.2	0.5	0.5	1.65
Pclass.3	-0.57	-0.57	0.57
EmbarkedC	0.48	0.48	1.62
EmbarkedQ	0.29	0.29	1.34
EmbarkedS	-0.03	-0.03	0.97
SibSp_norm	-0.26	-0.23	0.79
Parch_norm	-0.06	-0.08	0.92

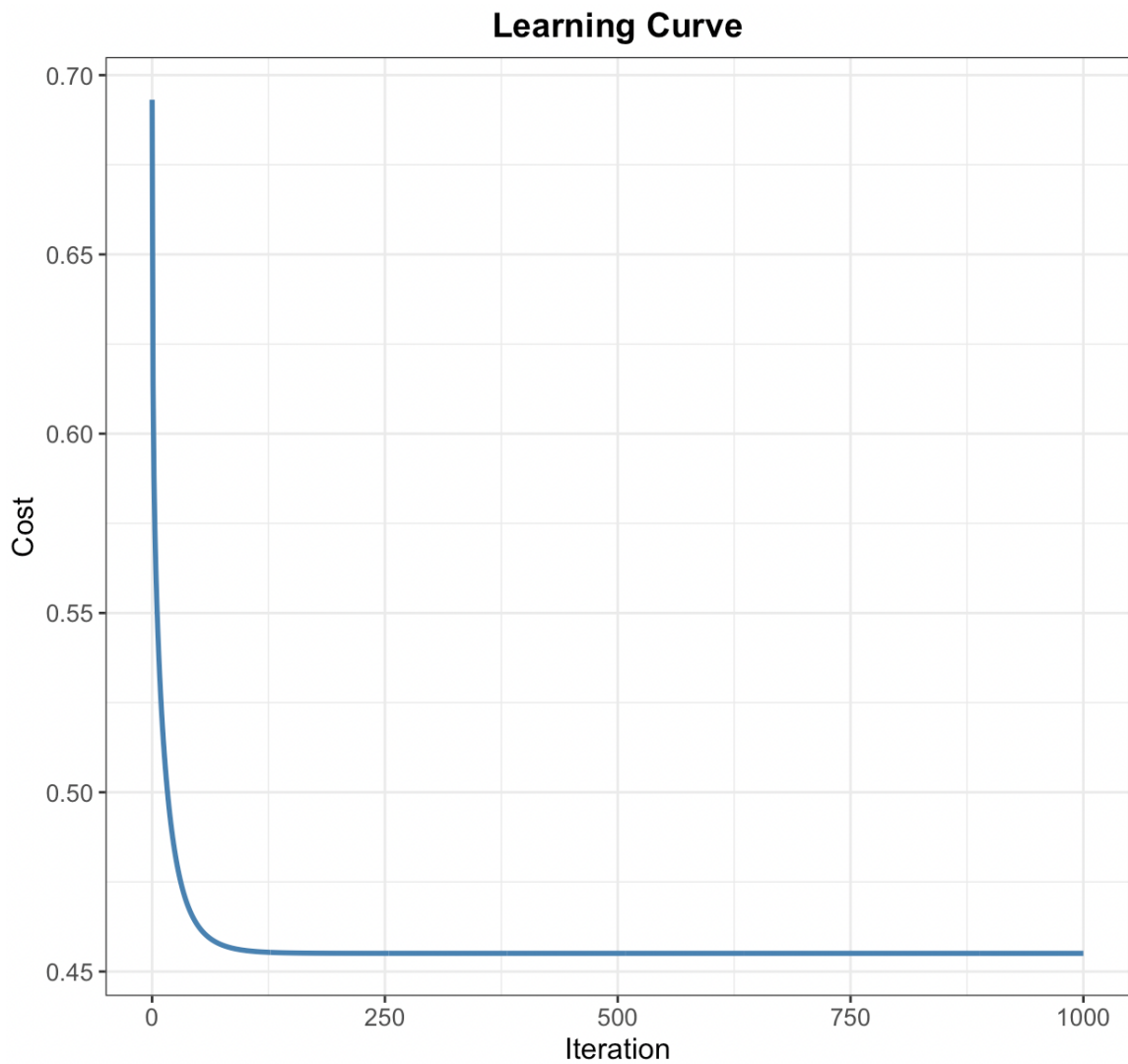
Fare_norm	0.15	0	1
-----------	------	---	---

[해석]

- **Sex** = 1 (남자)의 경우 생존확률이 0.07배 감소한다.
- **Pclass.1** 1등석의 경우 생존 확률이 2.84배 증가한다.
- **Pclass.3** 3등석의 경우 생존 확률이 0.57배 감소한다.
- **SibSp**, **Parch** 가 1씩 증가할수록 (함께 탑승한 형제, 배우자, 부모, 자식이 많아질수록) 생존 확률이 각 0.79, 0.92로 감소한다. 즉, 가족 구성원이 많을수록 생존 확률이 낮아진다.

10. 시각화

```
##%
costDF %>%
  ggplot(aes(x = iter, y = cost)) +
  geom_line(color = "steelblue", size = 1) +
  labs(
    title = "Learning Curve",
    x = "Iteration",
    y = "Cost"
  ) +
  theme_bw() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold",
    axis.title = element_text(size = 12),
    axis.text = element_text(size = 10)
  )
)
```



- iteration 100 이하의 구간에서 수렴

11. 성능 확인

```
# 예측 확률 저장
train_pred <- as.matrix(train_feature.df) %*% theta_vector

# 0.5를 기준으로 예측값을 0과 1로 변환
train_pred_class <- ifelse(train_pred >= 0.5, 1, 0)

conf_matrix <- confusionMatrix(factor(train_pred_class),
  factor(train_labelVector))
```

```
# 성능 지표 출력
cat("Train Accuracy:", conf_matrix$overall['Accuracy'], "\n")
```

- Train Accuracy: 0.81

Task5

- 문제 4에서 학습한 모델을 이용해서 test.csv 파일의 승객들에 대한 생존 여부를 예측하여라. 예측 결과를 gender_submission.csv와 같이 작성하여 kaggle에 제출하여 성능을 평가해보아라. train.csv의 예측성능과 비교해보고 overfitting이거나 underfitting인지 설명해보자

1. train과 똑같은 형태로 전처리

```
#### 필요한 열 선택
test <- test %>%
  select(Pclass, Sex, SibSp, Parch, Fare, Embarked)

#### one-hot encoding
# Sex 변수 처리
test$Sex <- ifelse(test$Sex == "male", 1, 0)

# Pclass를 범주형으로 변환
test$Pclass <- as.factor(test$Pclass)

# dummyVars 생성
dummy <- dummyVars(~ Pclass + Embarked, data = test)
# one-hot encoding 적용
encoded_df <- predict(dummy, newdata = test)

# 원본 데이터와 결합
test <- cbind(test, encoded_df)
# 원래의 범주형 변수 삭제
test <- test %>% select(-Pclass, -Embarked)

#### 표준화
test$SibSp_norm <- (test$SibSp - SibSp_mean) / SibSp_sd
test$Parch_norm <- (test$Parch - Parch_mean) / Parch_sd
```

```

test$Fare_norm <- (test$Fare - Fare_mean) / Fare_sd

# 표준화 이전 열 제거
test <- test[, -c(2:4)]

### feature matrix 생성
test_feature.df <- test %>%
  mutate(bias = 1) %>%
  select(bias, colnames(train)[2:11])

test_feature_matrix <- as.matrix(test_feature.df)

```

2. 예측

```

### test 예측 확률 저장
test_pred <- as.matrix(test_feature.df) %*% theta_vector

### 0.5를 기준으로 예측값을 0과 1로 변환
test_pred_class <- ifelse(test_pred >= 0.5, 1, 0)

### submission
submission$Survived <- test_pred_class
write.csv(submission, "submission.csv", row.names = FALSE)

```

3. 최종 결과



0.77033

- train accuracy 0.81에 비해 0.77로 약간 성능이 떨어졌지만, 그 차이가 크지 않기 때문에 과적합이 아니라고 판단했다.